SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	K subrahmanya v s manikanta	**Reg. No.:**	192472220
Section / Batch:	CSE-AI / 3rd year	**Date:**	

Code of Conduct

I K subrahmanya v s manikanta */ (192472220) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: K subrahmanya v s manikanta

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1

Morphology		
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as *connected*, *connection*, *connecting*, and *connectivity*, identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes *studies*, *studied*, *studying*, and *study*, identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players

5. restarting

6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.

2. Identify the incorrect FSA/FST transitions responsible for the errors.

3. Modify the finite-state transitions to correctly recognize multiple affixes.

4. Execute the corrected parser on the selected dataset.

5. Compare the parser output before and after correction.

6. Calculate the parsing accuracy before and after correction.

7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer

import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("news.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.

4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
    - Original text
    - Tokens
    - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
if word.endswith("s"):
    return word[:-2], "Plural Noun"
else:
    return word, "Singular"


words = [
  "cars",
  "boxes",
  "cities",
  "children",
  "books"
]


for w in words:
  print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.

2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
    - Regular plurals
    - Words ending in -es
    - Words ending in -ies
    - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer


stemmer = PorterStemmer()


documents = [

        "connected connection connecting connectivity",

        "studies studied studying study",

        "organize organized organizer organization"

]


vectorizer = CountVectorizer()


X = vectorizer.fit_transform(documents)

features = [

        stemmer.stem(word)

        for word in vectorizer.get_feature_names_out()
```

]

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy before correction
    - Classification accuracy after correction
    - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts

Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	3	3		75
2	3	3	3	3	3		
3	3	3	3	3	3		
4	3	3	3	3	3		
5	3	3	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
Dr. T Kumaragurubaran		
Signature: T Kumaragurubaran	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

The case describes a biomedical information-retrieval system that incorrectly decomposes treatment, treatable, retreatment, treated and untreated. The task requires identification of incorrect analyses, decomposition into prefixes, roots and suffixes, classification of affixes, root-cause analysis, correction, comparison and discussion of retrieval impact.

Morphological Decomposition

Word	Prefix	Root	Suffix	Morphological Classification
treatment	–	treat	-ment	Derivational suffix

treatable	–	treat	-able	Derivational suffix
retreatment	re-	treat	-ment	Derivational prefix + derivational suffix
treated	–	treat	-ed	Inflectional suffix
untreated	un-	treat	-ed	Derivational prefix + inflectional suffix

Word-by-Word Analysis

Treatment

Decomposition: treat + -ment. The root/base is 'treat'. The suffix '-ment' forms a noun from the verb 'treat' and is derivational because it creates a new lexical item and changes the word class.

Treatable

Decomposition: treat + -able. The suffix '-able' derives an adjective meaning capable of being treated. It is derivational because it changes the lexical category and meaning.

Retreatment

Decomposition: re- + treat + -ment. The prefix 're-' contributes the meaning 'again', while '-ment' derives a noun. Both affixes are derivational.

Treated

Decomposition: treat + -ed. Here '-ed' is an inflectional suffix marking past tense/past participle in the given form. It does not create a new lexical category.

Untreated

Decomposition: un- + treat + -ed. The prefix 'un-' creates the negative form and is derivational; '-ed' is inflectional in the past-participle/adjectival form.

Likely Sources of Error

1. Treating every suffix as inflectional without checking whether the suffix creates a new lexical item.
2. Removing derivational prefixes such as re- and un- and thereby losing important semantic information.
3. Failing to distinguish the lexical base 'treat' from derived forms such as treatment and treatable.
4. Using a simple suffix-removal rule that does not support multiple morphological layers.
5. Indexing only the final stem can cause semantically different biomedical terms to be merged.

Corrected Strategy

1. Use a lexicon containing valid roots and affixes.
2. Analyze possible derivational prefixes before inflectional suffixes.
3. Permit multiple affixes in a single word.
4. Store the root and morphological features separately from the surface word.
5. Do not remove semantically important derivational information during biomedical indexing.
6. Use normalized forms for retrieval while retaining morphological features for more precise matching.

3.5 Original vs Corrected Analysis

Word	Potential Incorrect Handling	Corrected Analysis
treatment	Treating -ment as ordinary inflection	treat + -ment; derivational noun formation
treatable	Reducing to treat without recording -able	treat + -able; derivational adjective
retreatment	Removing re- or treating it as	re- + treat + -ment; two

	noise	derivational layers
treated	Incorrectly treating -ed as derivational	treat + -ed; inflectional
untreated	Removing un- and losing negation	un- + treat + -ed; derivational + inflectional

3.6 Impact on Biomedical Information Retrieval

Correct morphological analysis can improve retrieval by preserving meaningful relationships among related biomedical terms. A system can recognize that treatment, retreatment and treated share the root 'treat' while retaining the additional information represented by derivational and inflectional affixes. This reduces inappropriate matches caused by over-aggressive stemming and can improve both recall and precision.

Question 2 – Error Analysis of Finite-State Morphological Parsing

The case describes a social-media NLP system whose finite-state morphological parser can recognize a single affix but fails when multiple prefixes and suffixes occur in the same word. The selected words are replayed, unhappier, disconnected, players, restarting and unreadable.

Correct Morphological Analyses

Word	Prefix	Root	Suffix	Features
replayed	re-	play	-ed	past tense / past participle
unhappier	un-	happy	-er	comparative
disconnected	dis-	connect	-ed	past tense / past participle
players	–	play	-er + -s	agentive derivation + plural
restarting	re-	start	-ing	progressive/present participle

unreadable	un-	read	-able	negative derivation + adjective formation

Incorrect FSA/FST Behavior

A parser that supports only one affix may successfully parse simple forms such as play + -ed, but fail when the word requires several transitions. For example, players requires play → -er → -s, while unhappier requires un- → happy → -er.

Problematic single-affix design:

```
START -> PREFIX -> ROOT -> SUFFIX -> END
```

Required multi-affix behavior:

```
START -> PREFIX* -> ROOT -> DERIVATIONAL_SUFFIX* -> INFLECTIONAL_SUFFIX* -> END
```

Corrected Transition Design

```
START
 |
 +--> PREFIX (optional, repeatable)
 |
+--> ROOT
      |
      +--> DERIVATIONAL SUFFIX (optional/repeatable)
      |
      +--> INFLECTIONAL SUFFIX (optional/repeatable)
      |
     END
```

Examples:

re + play + ed

un + happy + er

play + er + s

re + start + ing

un + read + able

Proposed Finite-State Rules

1. Allow zero or more derivational prefixes before the root.
2. Require a valid lexical root before suffix transitions.
3. Allow zero or more derivational suffixes after the root.
4. Allow appropriate inflectional suffixes after derivational suffixes.
5. Reject invalid affix sequences rather than forcing a partial parse.
6. Store each recognized morpheme and feature separately.

Sample Before/After Comparison

Word	Before Correction	After Correction
replayed	May recognize play or -ed only	re- + play + -ed
unhappier	May stop after un- or -er	un- + happy + -er
disconnected	May recognize dis- or -ed only	dis- + connect + -ed
players	May remove only -s	play + -er + -s
restarting	May recognize -ing only	re- + start + -ing
unreadable	May recognize un- only	un- + read + -able

Accuracy and Complexity

The source document asks for execution on Sentiment140 or another publicly available social-media dataset and calculation of parsing accuracy before and after correction. The assessment document does not provide the dataset output or measured accuracy values. Therefore, an exact dataset-level accuracy cannot be stated without running the parser against a labeled dataset.

For a controlled evaluation, accuracy can be calculated as:

```
Parsing Accuracy = Correctly Parsed Words / Total Evaluated Words × 100
```

If the parser has a fixed set of affix transitions, recognition can be efficient. Allowing multiple affixes increases the number of possible paths, so ambiguity may increase. Dynamic programming or memoization can be used to avoid repeatedly exploring the same states.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The original program reads news.csv and directly applies PorterStemmer.stem to the complete Text column. The assessment identifies that complete sentences are not being processed correctly and requires tokenization before stemming.

Original Program

```
from nltk.stem import PorterStemmer
import pandas as pd


ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

Errors Identified

1. The Porter stemmer is applied to an entire sentence instead of individual tokens.
2. The Text column is not tokenized before stemming.
3. Punctuation and other non-word characters are not explicitly handled.
4. The program assumes the file is named news.csv and contains a Text column.
5. The original pipeline does not preserve a clear token-level representation for comparison.

Corrected Pipeline

```
Raw document
    |
    v
Text normalization
    |
    v
Tokenization
    |
    v
Remove/handle punctuation
    |
    v
Porter stemming
    |
    v
Stemmed token sequence
    |
    v
Feature extraction / analysis
```

Corrected Python Program

```
import pandas as pd

import re

from nltk.stem import PorterStemmer

from nltk.tokenize import word_tokenize

import nltk



nltk.download("punkt", quiet=True)
```

```
nltk.download("punkt_tab", quiet=True)



ps = PorterStemmer()



# Read the news dataset

data = pd.read_csv("news.csv")



# Check that the required column exists

if "Text" not in data.columns:

    raise ValueError("CSV file must contain a 'Text' column.")



def preprocess(text):

    text = str(text).lower()



    # Tokenize complete sentences

    tokens = word_tokenize(text)



    # Keep alphabetic tokens

    tokens = [

        token for token in tokens

        if re.fullmatch(r"[a-zA-Z]+", token)

    ]



    stems = [

        ps.stem(token)

        for token in tokens

    ]
```

```
    return tokens, stems


data[["Tokens", "Stemmed_Tokens"]] = (
    data["Text"]
    .apply(preprocess)
    .apply(pd.Series)
)


print("\nORIGINAL TEXT / TOKENS / STEMS")
for _, row in data.head().iterrows():
    print("\nOriginal:", row["Text"])
    print("Tokens:", row["Tokens"])
    print("Stems:", row["Stemmed_Tokens"])
```

Example

Original Text	Tokens	Stemmed Tokens
The players are playing games.	the, players, are, playing, games	the, player, are, play, game
The students studied the topics.	the, students, studied, the, topics	the, student, studi, the, topic
Connected systems are connecting.	connected, systems, are, connecting	connect, system, are, connect

Twenty Problematic/Interesting Stemming Cases

Word	Porter Stem	Type / Observation
connected	connect	Inflectional; -ed removed
connection	connect	Derivational; -ion reduced
connecting	connect	Inflectional; -ing removed

connectivity	connect	Derivational normalization
studies	studi	Inflectional plural/verb form; stem is not dictionary word
studied	studi	Inflectional past form
studying	studi	Inflectional progressive form
study	studi	Base word can be altered by stemming
organize	organ	Potentially over-aggressive derivational reduction
organized	organ	Inflectional + derivational interaction
organizer	organ	Derivational -er removed
organization	organ	Derivational -ization/-ation reduced
relational	relat	Derivational suffix reduction
conditional	condit	Derivational reduction
rational	ration	Derivational reduction
cats	cat	Inflectional plural
boxes	box	Inflectional plural
running	run	Inflectional -ing with consonant adjustment
happiness	happi	Derivational -ness; stem not necessarily a dictionary word
fairness	fair	Derivational -ness

The table illustrates why stemming is not equivalent to true morphological analysis. A Porter stem can be useful for information retrieval because related forms may be reduced to a common representation,

but some outputs such as 'studi' or 'happi' are not dictionary words. The assessment therefore expects error identification rather than assuming every stem is linguistically perfect.

Improvement

Tokenizing before stemming ensures that the stemmer receives individual words rather than complete sentences. This makes the representation consistent, prevents sentence-level stemming errors, and creates a useful token-level feature representation for downstream NLP tasks.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The supplied parser is intended to identify plural nouns but uses word.endswith('s') and removes two characters. This is incorrect for regular plurals, -es plurals, -ies plurals and irregular forms.

Original Error

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

The expression word[:-2] removes two characters regardless of the plural rule. For 'cars', it produces 'car' correctly by accident only if the intended ending is treated carefully, but for 'books' it produces 'book' and for words ending in -es or -ies it does not represent the correct morphological transformation. It also cannot recognize irregular plurals such as 'children'.

Corrected Rules

Regular -s plurals: remove only the final s, e.g., books → book.

Plural -es forms: remove -es when the lexical base requires it, e.g., boxes → box.

Plural -ies forms: replace -ies with -y, e.g., cities → city.

Irregular plurals: use an explicit lexical dictionary, e.g., children → child.

Words not matching plural rules should remain singular.

Corrected Python Program

```
# Finite-state-style rule parser for common English plurals



IRREGULAR_PLURALS = {

    "children": "child",

    "men": "man",

    "women": "woman",

    "mice": "mouse",

    "geese": "goose",

    "feet": "foot",

    "teeth": "tooth",

    "people": "person"

}



def parser(word):

    w = word.lower()



    # Irregular plural

    if w in IRREGULAR_PLURALS:

        return IRREGULAR_PLURALS[w], "Plural Noun"



    # -ies -> -y

    if w.endswith("ies") and len(w) > 3:

        return w[:-3] + "y", "Plural Noun"



    # -es

    if w.endswith("es") and len(w) > 2:
```

```
    # Common cases such as boxes, buses, watches
    return w[:-2], "Plural Noun"

# Regular -s
if w.endswith("s") and not w.endswith("ss"):
    return w[:-1], "Plural Noun"

# Singular/default
return w, "Singular"


words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books",
    "car",
    "box",
    "city",
    "book"
]

for w in words:
    print(w, "->", parser(w))
```

Expected Corrected Output

Input	Corrected Base	Analysis

cars	car	Plural Noun
boxes	box	Plural Noun
cities	city	Plural Noun
children	child	Plural Noun – irregular
books	book	Plural Noun
car	car	Singular
box	box	Singular
city	city	Singular
book	book	Singular

Word Net-Based Extension

```
from nltk.corpus import wordnet as wn

import nltk



nltk.download("wordnet")



# WordNet can be used as a lexical resource to validate

# whether a candidate base form exists as an English word.



def exists_in_wordnet(word):
    return bool(wn.synsets(word))



for word in ["car", "box", "city", "child", "book"]:
    print(word, "->", exists_in_wordnet(word))
```

The source assessment asks for execution using WordNet or another public lexical resource. WordNet can support lexical validation, but it does not by itself provide a complete finite-state morphological grammar for every English plural. The explicit irregular dictionary and suffix rules therefore remain important.

Limitations of the Rule-Based Parser

1. English has many irregular plural forms that cannot be handled by simple suffix rules.
2. Some words ending in s are not plural nouns.
3. Some -es forms require lexical knowledge rather than a universal rule.
4. Ambiguous words can require sentence context.
5. A small hand-written rule set does not cover the full English morphology.
6. Rule order matters because multiple rules can potentially match the same word.